ASSIGNMENT - 18

Thumma Hasini | 2303A52076 | Batch - 37

API Integration: Connecting to External Services with Error Handling

Introduction

This lab focuses on integrating Python applications with external REST APIs. The goal is to understand how to send requests, process responses, and handle errors effectively. Each task demonstrates real-world API usage with proper validation and structured output.

Task 1 – Public Transport Route Fare API

Prompt Used

"Generate Python code using requests library to fetch fare details between two stations using a public transport API with proper error handling."

Explanation

In this task, we connect to a transport API to retrieve fare details between two stations.

- User inputs source and destination
- Input validation ensures valid station names
- API request is sent using requests.get()
- Errors like invalid stations, timeout, or API failure are handled using try-except blocks
- Output is displayed in a structured format

Code Screenshot

```
import requests

amount = float(input("Enter amount in INR: "))

try:
    url = "https://api.exchangerate-api.com/v4/latest/IN
    res = requests.get(url)

    if res.status_code == 200:
        data = res.json()
        rates = data["rates"]

        print("\nConverted Values:")
        print("USD:", amount * rates["USD"])
        print("EUR:", amount * rates["EUR"])
        print("GBP:", amount * rates["GBP"])
    else:
        print("API Error")

except:
    print("Error fetching data")
```

```
Enter destination: hyderabad
API failed

(.venv) C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding>




Enter destination: hyderabad
API failed
```

Task 2 – Currency Exchange Rates

Prompt Used

"Write Python code to convert INR to USD, EUR, and GBP using a currency exchange API with error handling and tabular output."

Explanation

This task converts Indian Rupees into multiple currencies.

- User inputs amount
- API fetches latest exchange rates
- Validations check for numeric input
- Errors such as API downtime or invalid responses are handled
- Results are displayed in a clean table format

Code Screenshot

Assignment_18.3.py > ...

```python
import requests

source = input("Enter source: ")
destination = input("Enter destination: ")

if not source or not destination:
    print("Invalid input")
else:
    try:
        url = f"https://api.example.com/fare?from=
        res = requests.get(url, timeout=5)

        if res.status_code == 200:
            data = res.json()
            print("Fare Details:")
            print("From:", source)
            print("To:", destination)
            print("Fare:", data.get("fare", "Not a
        else:
            print("Invalid station or API error")

    except:
        print("API failed")
```

```
Converted Values:
USD: 37.8102
EUR: 32.78073
GBP: 28.357650000000003

(.venv) C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding>
```

Task 3 – GitHub Repository Info Fetcher

Prompt Used

"Create a Python function to fetch GitHub repository details (stars, forks, issues) using GitHub API with error handling."

Explanation

This task connects to GitHub API to retrieve repository information.

- User provides repository name (owner/repo)
- API request fetches details
- Handles errors like:
    - 404 (repository not found)
    - Rate limit exceeded ○

      Invalid input
- Displays repository name, description, stars, forks, and issues

Code Screenshot

```python
import requests
repo = input("Enter repo (user/repo): ")
try:
    url = f"https://api.github.com/repos/{repo}"
    res = requests.get(url)

    if res.status_code == 200:
        data = res.json()
        print("\nRepo Info:")
        print("Name:", data["name"])
        print("Description:", data["description"])
        print("Stars:", data["stargazers_count"])
        print("Forks:", data["forks_count"])
        print("Issues:", data["open_issues_count"])

    elif res.status_code == 404:
        print("Repository not found")

    else:
        print("Error:", res.status_code)

except:
    print("Network error")
```

```
Enter repo (user/repo): AI Assisted coding
Enter repo (user/repo): AI Assisted coding
Repository not found
PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding>
```

Task 4 – Real-Time Application: News Headlines Aggregator

Prompt Used

"Generate Python code to fetch top 5 news headlines by category using a news API with retry mechanism and error handling."

Explanation

This task builds a simple news aggregator.

- User selects category (sports, technology, health)
- API fetches top headlines
- Validates category input
- Implements retry logic if request fails
- Handles errors like:
    - Invalid API key
    - Network failure
- Displays headlines in numbered format

Code Screenshot

```
signment_18.3.py > ...
import requests

api_key = "YOUR_API_KEY"
category = input("Enter category (sports/technology/h

url = f"https://newsapi.org/v2/top-headlines?category

for i in range(2):  # retry 2 times
    try:
        res = requests.get(url)

        if res.status_code == 200:
            data = res.json()
            articles = data["articles"][:5]

            print("\nTop Headlines:")
            for i, art in enumerate(articles, 1):
                print(f"{i}. {art['title']}")
            break

        else:
            print("Invalid category or API error")

    except:
        print("Retrying...")
```

```
Enter category (sports/technology/health): health
Invalid category or API error
Invalid category or API error
PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding>
```

Task 5 – COVID-19 Statistics API Integration

Prompt Used

"Write Python code using requests to fetch COVID-19 statistics for a given country with error handling and retry logic."

Explanation

This task retrieves COVID-19 statistics for a specific country. •

User inputs country name

- API returns:
  - Total confirmed cases
  - Total deaths ○ Total recovered ○ Active cases
- Handles errors:
  - Invalid country name
  - API rate limit exceeded ○ Network/server issues
- Retry logic is implemented for rate limits
- Displays data clearly

Code Screenshot

```python
import requests
country = input("Enter country: ")
try:
    url = f"https://disease.sh/v3/covid-19/countries/{c
    res = requests.get(url)
    if res.status_code == 200:
        data = res.json()

        print("\nCOVID Stats:")
        print("Cases:", data["cases"])
        print("Deaths:", data["deaths"])
        print("Recovered:", data["recovered"])
        print("Active:", data["active"])

    elif res.status_code == 404:
        print("Invalid country")

    else:
        print("API Error")
except:
    print("Network error")
```

```
Enter country: india

COVID Stats:
Cases: 45035393
Deaths: 533570
Recovered: 0
Active: 44501823
PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding>
```

Conclusion In this lab, we successfully learned how to integrate Python applications with

external APIs. We explored multiple real-world scenarios such as transport systems, currency conversion, GitHub data retrieval, news aggregation, and COVID-19 statistics.

Key takeaways:

- APIs allow real-time data integration into applications
- Proper error handling is essential for reliability
- Input validation prevents unexpected failures
- Retry mechanisms improve robustness in unstable conditions

This lab strengthened practical skills in API handling, making applications more dynamic, reliable, and user-friendly.